

ChronoSmart: RTC-Based Appliance Automation & Temperature Monitor

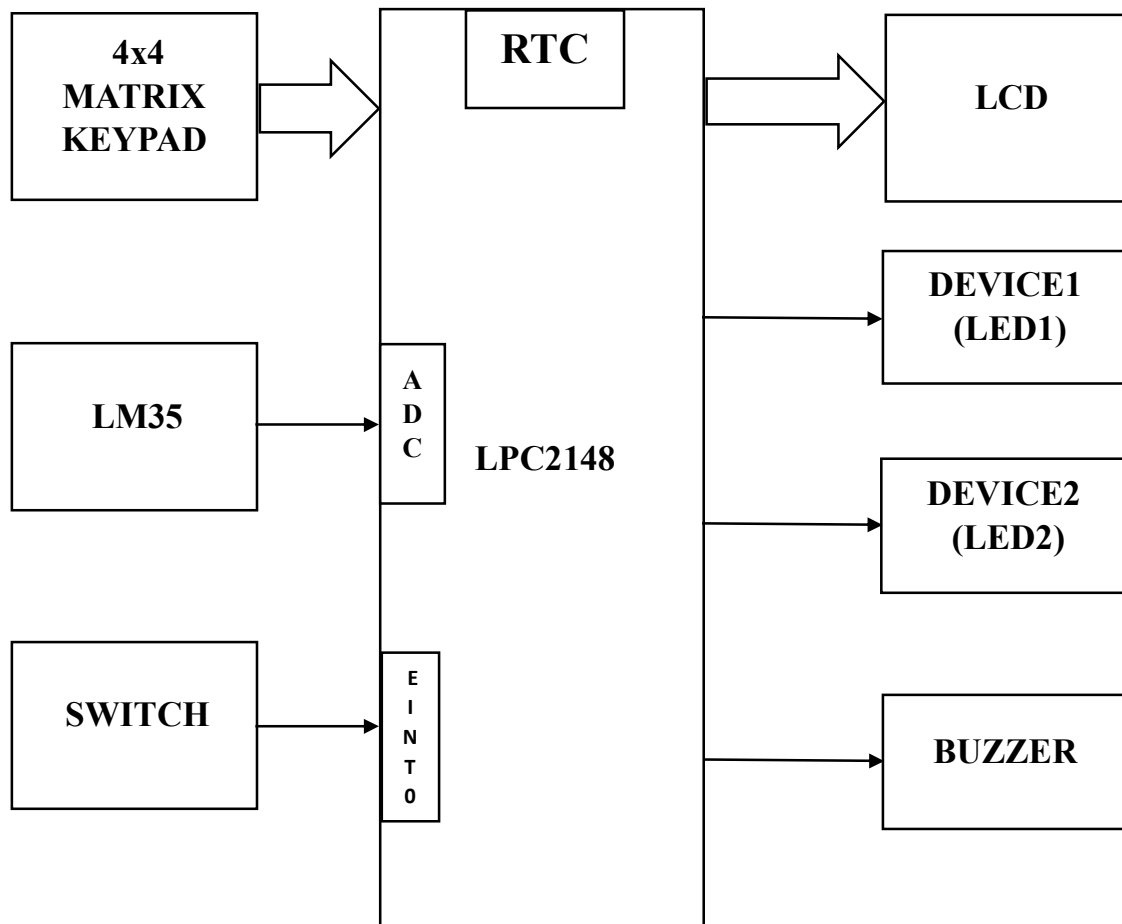
AIM:

To design and develop a real-time embedded system using the LPC2148 that automates appliance control based on user-defined time schedules while simultaneously monitoring ambient temperature using on-chip ADC.

Objectives:

- Implement RTC-based scheduling for automatic appliance control
- Interface keypad for user input of ON/OFF timings
- Monitor temperature using ADC (LM35 sensor)
- Control appliances using GPIO (LED/Relay)

BLOCK DIAGRAM:



REQUIREMENTS:

HARDWARE REQUIREMENTS:

- LPC2148
- 16X2 LCD
- 4X4 MATRIX KEYPAD
- LED's
- SWITCH
- LM35
- BUZZER
- USB-UART CONVERTER / DB-9 CABLE

SOFTWARE REQUIREMENTS:

- EMBEDDED-C PROGRAMMING
- FLASH MAGIC

PROJECT WORKFLOW:

ChronoSmart is a real-time embedded system that automates appliance control based on RTC scheduling while continuously monitoring temperature using ADC on LPC2148.

Initialize all required peripherals and start the application logic. In continuous loop, read the RTC information such as Hour, Minute, Second, Day, Date, Month, and Year, and display it on the LCD. Simultaneously read the temperature through ADC and update the display. Maintain independent ON and OFF scheduling parameters for two different devices. Compare the current RTC time with the scheduled ON/OFF timings of Device-1 and Device-2 and control each device accordingly.

To modify any system parameter, the user must generate an interrupt request through a dedicated switch. On receiving the interrupt request, the system enters secure edit mode only after successful password authentication using the keypad. After authentication, the user can edit RTC settings, Device-1 timing, Device-2 timing, temperature

threshold, and other required parameters. If invalid password attempts exceed the allowed limit, the system denies access and temporarily locks the edit mode. After editing, the system returns to continuous monitoring and control operation.

System Working:

Initialize all required peripherals and start the application logic. The required peripherals include:

- RTC
- LCD
- GPIO for Device-1 and Device-2 control
- Switch input
- External Interrupt / Interrupt source for user request
- Keypad for user input
- ADC for temperature monitoring
- Buzzer or status LED for alert/security indication

After initialization, the system enters continuous monitoring mode.

Continuous Monitoring Operation

Inside the infinite loop, the controller performs the following operations continuously:

Read and Display RTC & ADC Information on LCD

Read the current RTC parameters: Hour, Minute, Second, Day, Date, Month and Year

Display the RTC information on the LCD in formatted form.

Read the analog temperature sensor value through ADC, convert it into temperature, and display it on the LCD along with system status.

Example display: HH:MM:SS T:28°C

DD/MM/YYYY DAY

Control Two Devices Independently

The system maintains separate scheduling parameters for two devices.

For each device, store:

- ON time
- OFF time
- Enable/Disable status

Example:

- Device-1 ON time = 08:00:00
- Device-1 OFF time = 09:00:00
- Device-2 ON time = 10:30:00
- Device-2 OFF time = 11:15:00

The controller continuously compares current RTC time with the stored schedule.

Device-1 Logic

- If current time is greater than or equal to Device-1 ON time, turn ON Device-1
- If current time is greater than or equal to Device-1 OFF time, turn OFF Device-1

Device-2 Logic

- If current time is greater than or equal to Device-2 ON time, turn ON Device-2
- If current time is greater than or equal to Device-2 OFF time, turn OFF Device-2

Each device works independently based on its own configured schedule.

Interrupt-Based User Editing:

Instead of continuously polling for edit mode, the system uses an **interrupt request** to enter configuration mode.

Interrupt Request for User Editing

When the user presses the interrupt-based request switch, an interrupt occurs.

On interrupt:

- Set an edit_request_flag
- Main logic checks this flag
- System temporarily enters secure edit mode

This approach is better because:

- normal monitoring remains uninterrupted
- user editing is event-driven
- students learn interrupt-based design

Basic Level Security Inclusion

Before allowing any modification, the system asks for a basic password.

Security Check Before Editing

When interrupt request is received:

- Prompt user to enter password through keypad
- Compare entered password with stored password
- If password is correct:
 - allow editing of parameters
- If password is wrong:
 - deny access
 - increment wrong-attempt counter

- show “Access Denied” on LCD
- optionally blink buzzer/LED

Wrong Attempt Handling

- If wrong password is entered 3 times:
 - lock edit mode for a short duration or until reset
 - show “System Locked”
 - optional alert indication through buzzer

This gives the project a simple but attractive security layer.

Editable Parameters in Secure Edit Mode

After successful password validation, allow user to edit:

- Current RTC time and date
- Device-1 ON time
- Device-1 OFF time
- Device-2 ON time
- Device-2 OFF time
- Device enable/disable option
- Temperature threshold limit
- Password change option

The keypad can be used to navigate through a menu.

Return to Normal Mode

After editing is completed:

- save updated parameters
- clear edit request flag
- return to continuous monitoring mode

The system then resumes:

- RTC display
- temperature monitoring
- scheduled control of both devices

***** ALL THE BEST *****

Note:

The application code must be developed following industry-standard embedded programming practices, with proper structure, naming conventions, modular design, and complete test case validation support. Any code not adhering to these standards will not be accepted.